

person_state_manager.py 파일 정리

전체 코드

```
# person_state_manager.py
import time
class PersonStateManager:
    def __init__(self, disappear_timeout=3.0):
        self.person_states = {}
        self.disappear_timeout = disappear_timeout
    def create_person_state(self):
        now = time.time()
        return {
            "first_seen": now,
            "last_seen": now,
            "bbox": None,
            "is_full_body": False,
            "crop_saved": False,
            "crop_path": None,
            "is_recording": False,
            "clip_path": None,
            "vlm_done": False,
            "vlm_result": None,
        }

    def update_person(self, person_id, bbox, is_full_body):

        now = time.time()

        if person_id not in self.person_states:
            self.person_states[person_id] = self.create_person_state()

        state = self.person_states[person_id]

        state["last_seen"] = now
        state["bbox"] = bbox
        state["is_full_body"] = is_full_body

        return state

    def mark_crop_saved(self, person_id, crop_path):

        if person_id not in self.person_states:
            self.person_states[person_id] = self.create_person_state()
```

```

self.person_states[person_id]["crop_saved"] = True
self.person_states[person_id]["crop_path"] = crop_path

def mark_recording_started(self, person_id, clip_path=None):

    if person_id not in self.person_states:
        self.person_states[person_id] = self.create_person_state()

    self.person_states[person_id]["is_recording"] = True
    self.person_states[person_id]["clip_path"] = clip_path

def mark_recording_stopped(self, person_id):

    if person_id in self.person_states:
        self.person_states[person_id]["is_recording"] = False

def mark_vlm_done(self, person_id, vlm_result):
    """
    VLM 분석 완료 상태 기록
    """

    if person_id not in self.person_states:
        self.person_states[person_id] = self.create_person_state()

    self.person_states[person_id]["vlm_done"] = True
    self.person_states[person_id]["vlm_result"] = vlm_result

def get_state(self, person_id):
    return self.person_states.get(person_id)

def has_crop_saved(self, person_id):
    state = self.get_state(person_id)
    return state is not None and state["crop_saved"]

def is_recording(self, person_id):
    state = self.get_state(person_id)
    return state is not None and state["is_recording"]

def is_vlm_done(self, person_id):
    state = self.get_state(person_id)
    return state is not None and state["vlm_done"]

def remove_disappeared_persons(self):
    now = time.time()
    removed_ids = []

```

```

    for person_id, state in list(self.person_states.items()):
        if now - state["last_seen"] > self.disappear_timeout:
            removed_ids.append(person_id)
            del self.person_states[person_id]

    return removed_ids

```

클래스 역할

사람마다 상태를 기억하는 관리자 역할

- person_id마다 crop저장 여부, VLM분석 여부, 최종 탐지 시점등을 기억하는 클래스

함수별 역할

__init().__

- id별 상태 저장 딕셔너리 생성

```

{
    1: {...},
    2: {...}
}

```

- 사람 몇초동안 안보이면 사라진 사람으로 판단할건지 타임아웃 지정

create_person_state()

- 신규 상태 생성후 리턴 역할
- 상태값
 - 처음/마지막 등장시간
 - 전신 보였는지 여부
 - crop저장 여부, 저장 경로
 - VLM분석 여부, 결과 저장
 - 클립 녹화 여부, 저장 경로

update_person()

- YOLO가 사람 탐지할때마다 호출해서 상태 갱신
- id 딕셔너리에 없는 사람(처음 탐지된 사람)이면 create_person_state()로 상태 생성
- 마지막 등장시간, 바운딩 박스 좌표(현재 위치), 전신 확인 여부, 갱신 후 상태 리턴

mark_crop_saved()

- crop저장후 crop저장 상태와 경로 업데이트하는 함수
- 저장 성공여부는 호출부에서 처리

mark_vlm_done()

- VLM분석 완료 처리

get_state()

- 상태 가져오는 함수
- .get()은 딕셔너리 자체함수 - 없으면 None반환

has_crop_saved()

- state가 존재하고 crop이 저장되어있으면 True리턴

is_vlm_done()

- state가 존재하고 VLM 분석되었으면 True리턴

remove_disappeared_persons()

- 일정시간 보이지 않은 사람 메모리에서 제거
- 딕셔너리에 id값 포화 방지, 상태 누적 저장으로 꼬임 방지, ID재사용 문제등 방지 용도

예시)

12초에 사람이 마지막으로 보였는데 지금이 16초면 안보인지 4초 경과
근데 timeout이 3초로 셋팅되어있으면 해당 person_id 메모리에서 제거.

mark_recording_started(), mark_recording_stopped(), is_recording()는 클립 영상 저장 관련 함수
인데 아직 쓰일지 안쓰일지 몰라서 아직은 따로 정리 안할게요.